In [1]:

```
# MantisV2 reproduction - datasets 7-12 (Tables 2 and 3), Padmanabh Khunt (26100085)
# Install the official Mantis package + data download helpers
!pip install -q mantis-tsfm gdown h5py
!nproc; free -g | head -2; df -h /tmp /kaggle/working | tail -2
```

```
4
               total        used        free      shared  buff/cache
available
Mem:              31           1          26           0           4
30
overlay          8.0T  7.1T  967G  89% /
/dev/loop2        20G  140K   20G   1% /kaggle/working
```

MantisV2 reproduction: datasets 7-12 (Tables 2 and 3)

AI for Time Series group project, Padmanabh Khunt (26100085).

Paper: MantisV2: Closing the Zero-Shot Gap in Time Series Classification with Synthetic Data and Test-Time Strategies (Feofanov et al., 2026, arXiv:2602.17868). Code: https://github.com/vfeofanov/mantis (https://github.com/vfeofanov/mantis) (pip package mantis-tsfm), checkpoint paris-noah/MantisV2 on HuggingFace.

My datasets (7-12 when Tables 2 and 3 are read in order): UCI-HAR, Blink, CAP-ID, CAP-OOD, Epilepsy-EEG, FingerMovements.

Protocol used in the paper (zero-shot feature extraction):

1. Resize every series to length 512 with linear interpolation.
2. Frozen MantisV2 encoder, output of transformer layer 2 (0-indexed), output token = concat(cls token, mean of the other tokens), 512 dims per channel.
3. Each channel is encoded independently and the embeddings are concatenated.
4. Random Forest, 200 trees, unlimited depth, trained on the train embeddings, accuracy on the test set.
5. Mean and std over 3 seeds.

Run on Kaggle (CPU session, 4 cores, 31 GB RAM). The first version of this notebook ran on Google Colab (T4), where UCI-HAR, Blink, Epilepsy-EEG and FingerMovements were reproduced before the Colab quota ran out.

In [2]:

```
import os, time, json, zipfile, subprocess
import numpy as np
import pandas as pd
import requests
import torch
import torch.nn.functional as F
from sklearn.ensemble import RandomForestClassifier
from mantis.architecture import MantisV2
from mantis.trainer import MantisTrainer

device = 'cuda' if torch.cuda.is_available() else 'cpu'
print('device:', device, '| torch threads:', torch.get_num_threads())

DATA = '/tmp/data'          # large scratch disk (CAP needs about 19 GB w
hile unzipping)
OUT = '/kaggle/working'     # saved with the notebook version (20 GB limi
t)
os.makedirs(DATA, exist_ok=True)

# MantisV2 column of Tables 2 and 3 in the paper (mean, std over 3 seeds)
PAPER = {
    'UCI-HAR':         (0.9013, 0.002),
    'Blink':           (0.9956, 0.002),
    'CAP-ID':          (0.8152, 0.001),
    'CAP-OOD':         (0.7859, 0.000),
    'Epilepsy-EEG':    (0.9558, 0.001),
    'FingerMovements': (0.5500, 0.010),
}
results = {}
```

```
device: cpu | torch threads: 2
```

Load the pre-trained MantisV2 encoder

The repo README says the best zero-shot setting for MantisV2 is layer index 2 with output_token='combined'. The repo tutorial (getting_started/intermediate_layers.ipynb) sets both attributes again after from_pretrained, so we do the same and check the embedding size (should be 512 per channel).

In [3]:

```
LAYER, TOKEN = 2, 'combined'

network = MantisV2(device=device, return_transf_layer=LAYER, output_token
=TOKEN)
network = network.from_pretrained('paris-noah/MantisV2')
network.return_transf_layer, network.output_token = LAYER, TOKEN  # make
sure loading did not reset them

model = MantisTrainer(device=device, network=network)
print('parameters: %.2fM' % (sum(p.numel() for p in network.parameters())
/ 1e6))
print('embedding size per channel:', model.transform(np.random.randn(4,
1, 512).astype(np.float32)).shape[1])
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Pleas
e set a HF_TOKEN to enable higher rate limits and faster downloads.
```

```
parameters: 4.19M
embedding size per channel: 512
```

In [4]:

```
def resize(X, length=512):
    """Linear interpolation to the length the model was pre-trained o
n."""
    return F.interpolate(torch.tensor(X, dtype=torch.float), size=length,
mode='linear', align_corners=False).numpy()

def embed(X, batch_size=256):
    """Frozen MantisV2 features; channels are encoded independently and c
oncatenated."""
    X = np.nan_to_num(np.asarray(X, dtype=np.float32))
    return model.transform(resize(X), batch_size=batch_size)

def evaluate(name, Z_train, y_train, Z_test, y_test, embed_s=0.0, seeds=
(0, 1, 2)):
    """Paper protocol: Random Forest (200 trees, unlimited depth) on froz
en embeddings, 3 seeds."""
    accs, t0 = [], time.time()
    for seed in seeds:
        rf = RandomForestClassifier(n_estimators=200, max_depth=None, n_j
obs=-1, random_state=seed)
        rf.fit(Z_train, y_train)
        accs.append(float(np.mean(rf.predict(Z_test) == np.asarray(y_tes
t))))
        print(f'  seed {seed}: accuracy {accs[-1]:.4f}', flush=True)
    paper_mean, paper_std = PAPER[name]
    results[name] = dict(ours_mean=np.mean(accs), ours_std=np.std(accs),
paper_mean=paper_mean, paper_std=paper_std,
                         diff=np.mean(accs) - paper_mean, per_seed=json.d
umps(accs), n_train=len(y_train),
                         n_test=len(y_test), feat_dim=Z_train.shape[1], e
mbed_s=round(embed_s, 1),
                         rf_s=round(time.time() - t0, 1))
    pd.DataFrame(results).T.to_csv(f'{OUT}/mantisv2_results_datasets_7_1
2.csv')  # saved after every dataset
    print(f'{name}: ours {np.mean(accs):.4f} +/- {np.std(accs):.4f} | pap
er {paper_mean:.4f} +/- {paper_std:.3f} | '
          f'diff {np.mean(accs) - paper_mean:+.4f} | RF {time.time() - t
0:.0f}s', flush=True)

def evaluate_raw(name, X_train, y_train, X_test, y_test):
```

```
    print(f'{name}: train {X_train.shape}, test {X_test.shape}, classes
{len(np.unique(y_train))}')
    t0 = time.time()
    Z_train, Z_test = embed(X_train), embed(X_test)
    evaluate(name, Z_train, y_train, Z_test, y_test, time.time() - t0)

def safe(fn):
    """Run one dataset; a download problem on one dataset must not stop t
he rest of the run."""
    try:
        fn()
    except Exception as e:
        print('FAILED:', type(e).__name__, e)
```

Datasets 12 and 8: FingerMovements (UEA) and Blink

Both come from timeseriesclassification.com (the paper downloads Blink with the UniTS script, which uses the same zip). The .ts files are parsed directly: on current Colab images the aeon loader fails because numba does not support NumPy 2.4. Paper Table 6: FingerMovements 28 channels, length 50, train 316, test 100, 2 classes. Blink 4 channels, length 510, train 500, test 450, 2 classes.

In [5]:

```
def load_ts_archive(name):
    """Download <name>.zip from timeseriesclassification.com and parse th
e equal-length .ts files."""
    folder = f'{DATA}/{name}'
    if not os.path.exists(folder):
        subprocess.run(['wget', '-q', '-O', f'{DATA}/{name}.zip',
                        f'https://www.timeseriesclassification.com/aeon-t
oolkit/{name}.zip'], check=True)
        with zipfile.ZipFile(f'{DATA}/{name}.zip') as z:
            z.extractall(folder)
    out = {}
    for split in ['TRAIN', 'TEST']:
        path = [os.path.join(r, f) for r, _, fs in os.walk(folder) for f
in fs if f == f'{name}_{split}.ts'][0]
        X, y, in_data = [], [], False
        with open(path, encoding='utf-8', errors='ignore') as fh:
            for line in fh:
                line = line.strip()
                if not line or line.startswith('#'):
                    continue
                if line.lower().startswith('@data'):
                    in_data = True
                    continue
                if in_data:
                    *channels, label = line.split(':')
                    X.append([[float(v) if v != '?' else np.nan for v in
ch.split(',')] for ch in channels])
                    y.append(label)
        out[split.lower()] = (np.asarray(X, dtype=np.float32), np.asarray
(y))
    return out

def run_archive(name):
    d = load_ts_archive(name)
    evaluate_raw(name, *d['train'], *d['test'])

safe(lambda: run_archive('FingerMovements'))
safe(lambda: run_archive('Blink'))
```

```
FingerMovements: train (316, 28, 50), test (100, 28, 50), classes 2
  seed 0: accuracy 0.5400
  seed 1: accuracy 0.5600
  seed 2: accuracy 0.5500
FingerMovements: ours 0.5500 +/- 0.0082 | paper 0.5500 +/- 0.010 | dif
f +0.0000 | RF 6s
Blink: train (500, 4, 510), test (450, 4, 510), classes 2
  seed 0: accuracy 0.9978
  seed 1: accuracy 0.9956
  seed 2: accuracy 0.9933
Blink: ours 0.9956 +/- 0.0018 | paper 0.9956 +/- 0.002 | diff -0.0000
| RF 2s
```

Datasets 7 and 11: UCI-HAR and Epilepsy-EEG (TF-C preprocessing)

The paper says both are preprocessed following Zhang et al. (2022), i.e. the TF-C paper. TF-C publishes the preprocessed train/val/test splits on figshare (github.com/mims-harvard/TFC-pretraining). The figshare zip links answer scripts with a bot-check page, so the files are fetched through the official figshare API. We train on train.pt and test on test.pt (val.pt unused). Paper Tables 5/6: UCI-HAR 3 channels, length 206, train 5881, test 2947, 6 classes. Epilepsy-EEG 1 channel, length 178, train 60, test 11420, 2 classes.

In [6]:

```
TFC = {'HAR': 19930244, 'Epilepsy': 19930199}  # figshare article ids

def load_tfc(name):
    folder = f'{DATA}/{name}'
    os.makedirs(folder, exist_ok=True)
    files = requests.get(f'https://api.figshare.com/v2/articles/{TFC[nam
e]}', timeout=30).json()['files']
    out = {}
    for split in ['train', 'test']:
        path = f'{folder}/{split}.pt'
        if not os.path.exists(path):
            url = [f['download_url'] for f in files if f['name'] == f'{sp
lit}.pt'][0]
            with open(path, 'wb') as fh:
                fh.write(requests.get(url, timeout=600).content)
        d = torch.load(path, weights_only=False)
        X, y = np.asarray(d['samples'], dtype=np.float32), np.asarray(d
['labels'])
        if X.ndim == 2:
            X = X[:, None, :]
        out[split] = (X, y)
    return out

def run_tfc(folder, name):
    d = load_tfc(folder)
    evaluate_raw(name, *d['train'], *d['test'])

safe(lambda: run_tfc('HAR', 'UCI-HAR'))
safe(lambda: run_tfc('Epilepsy', 'Epilepsy-EEG'))
```

```
UCI-HAR: train (5881, 3, 206), test (2947, 3, 206), classes 6
  seed 0: accuracy 0.9033
  seed 1: accuracy 0.8999
  seed 2: accuracy 0.9006
UCI-HAR: ours 0.9013 +/- 0.0015 | paper 0.9013 +/- 0.002 | diff -0.000
0 | RF 57s
Epilepsy-EEG: train (60, 1, 178), test (11420, 1, 178), classes 2
  seed 0: accuracy 0.9542
  seed 1: accuracy 0.9561
  seed 2: accuracy 0.9570
Epilepsy-EEG: ours 0.9558 +/- 0.0012 | paper 0.9558 +/- 0.001 | diff -
0.0000 | RF 1s
```

Datasets 9 and 10: CAP-ID and CAP-OOD (WOODS benchmark)

Sleep-stage classification from the CAP database, preprocessed by WOODS (Gagnon-Audet et al., 2022). We use the WOODS preprocessed download (Google Drive link in woods/download.py, github.com/jc-audet/WOODS): one h5 file with a group per recording machine, MachineK/data (n, 3000, 19) and MachineK/labels (n, 1). Paper Table 6: 19 channels, length 3000, 6 classes. CAP-ID train 25748, test 10098. CAP-OOD train 27393, test 8265. Paper Appendix A.1.3: ID = merge all domains, then a 63.75 / 11.25 / 25 train/val/test split. OOD = Machines 0, 1, 3 train, Machine 2 validation, Machine 4 test.

In [7]:

```
import h5py, shutil, gdown
print('free disk in /tmp: %.0f GB' % (shutil.disk_usage('/tmp').free / 1e
9))

def get_cap():
    if not os.path.exists(f'{DATA}/CAP'):
        gdown.download(id='1NFwX2CqLrenWF4az0c6J-OglAoD48PAT', output=
f'{DATA}/cap.zip', quiet=True)
        with zipfile.ZipFile(f'{DATA}/cap.zip') as z:
            z.extractall(f'{DATA}/CAP')
        os.remove(f'{DATA}/cap.zip')
    return [os.path.join(r, f) for r, _, fs in os.walk(f'{DATA}/CAP') for
f in fs if f.endswith('.h5')][0]

t0 = time.time()
cap_file = get_cap()
print(f'CAP ready in {time.time() - t0:.0f}s:', cap_file)
with h5py.File(cap_file, 'r') as h:
    h.visititems(lambda name, obj: print(name, getattr(obj, 'shape', ''),
getattr(obj, 'dtype', '')))
```

```
free disk in /tmp: 1038 GB
CAP ready in 180s: /tmp/data/CAP/CAP.h5
Machine0
Machine0/data (12769, 3000, 19) float32
Machine0/labels (12769, 1) int64
Machine1
Machine1/data (4694, 3000, 19) float32
Machine1/labels (4694, 1) int64
Machine2
Machine2/data (4732, 3000, 19) float32
Machine2/labels (4732, 1) int64
Machine3
Machine3/data (9930, 3000, 19) float32
Machine3/labels (9930, 1) int64
Machine4
Machine4/data (8265, 3000, 19) float32
Machine4/labels (8265, 1) int64
```

In [8]:

```
# Each machine is embedded once, in chunks so the raw 3000-step signals n
ever all sit in RAM.
# The embeddings are reused for both splits and saved to the output (for
the improvement experiments later).
def embed_cap_machine(h, machine, chunk=2000):
    data, Z = h[f'{machine}/data'], []
    for start in range(0, data.shape[0], chunk):
        X = np.transpose(data[start:start + chunk], (0, 2, 1))  # (n, 300
0, 19) -> (n, 19, 3000)
        Z.append(embed(X))
    return np.concatenate(Z), h[f'{machine}/labels'][:].ravel()

cap_Z, cap_y = {}, {}
t0 = time.time()
with h5py.File(cap_file, 'r') as h:
    for m in sorted(h.keys()):
        cap_Z[m], cap_y[m] = embed_cap_machine(h, m)
        np.savez(f'{OUT}/cap_embeddings_{m}.npz', Z=cap_Z[m], y=cap_y[m])
        print(f'{m}: {cap_Z[m].shape}, classes {np.bincount(cap_y[m])},
{time.time() - t0:.0f}s', flush=True)
cap_embed_s = time.time() - t0
```

```
Machine0: (12769, 9728), classes [1448  350 4986 1533 2110 2342], 1048
s
Machine1: (4694, 9728), classes [ 318  171 1933  595  706  971], 1449s
Machine2: (4732, 9728), classes [1318  294 1168  595  810  547], 1850s
Machine3: (9930, 9728), classes [1114  580 3547 1273 1606 1810], 2684s
Machine4: (8265, 9728), classes [ 967  276 3377  711 1251 1683], 3386s
```

CAP-ID split

The paper does not say how the 63.75 / 11.25 / 25 split was randomised (seed, stratification). We use a plain random permutation with seed 0. The sizes match the paper exactly (train 25748, test 10098), but the exact samples may differ, which is a possible source of small differences.

In [9]:

```
machines = sorted(cap_Z)
Z_all = np.concatenate([cap_Z[m] for m in machines])
y_all = np.concatenate([cap_y[m] for m in machines])
n = len(y_all)
perm = np.random.RandomState(0).permutation(n)
n_train, n_test = int(0.6375 * n), int(round(0.25 * n))
tr, te = perm[:n_train], perm[n - n_test:]
print('total', n, '| train', len(tr), '| val (unused)', n - n_train - n_t
est, '| test', len(te))
evaluate('CAP-ID', Z_all[tr], y_all[tr], Z_all[te], y_all[te], cap_embed_
s)
del Z_all
```

```
total 40390 | train 25748 | val (unused) 4544 | test 10098
  seed 0: accuracy 0.8106
  seed 1: accuracy 0.8086
  seed 2: accuracy 0.8082
CAP-ID: ours 0.8091 +/- 0.0010 | paper 0.8152 +/- 0.001 | diff -0.0061
| RF 1596s
```

In [10]:

```
# CAP-OOD: Machines 0, 1, 3 for training, Machine 2 = validation (unuse
d), Machine 4 = test
train_m, test_m = ['Machine0', 'Machine1', 'Machine3'], ['Machine4']
Z_tr = np.concatenate([cap_Z[m] for m in train_m]); y_tr = np.concatenate
([cap_y[m] for m in train_m])
Z_te = np.concatenate([cap_Z[m] for m in test_m]);  y_te = np.concatenate
([cap_y[m] for m in test_m])
print('train', len(y_tr), '| test', len(y_te))
evaluate('CAP-OOD', Z_tr, y_tr, Z_te, y_te, cap_embed_s)
```

```
train 27393 | test 8265
  seed 0: accuracy 0.7851
  seed 1: accuracy 0.7840
  seed 2: accuracy 0.7883
CAP-OOD: ours 0.7858 +/- 0.0018 | paper 0.7859 +/- 0.000 | diff -0.000
1 | RF 1681s
```

Results vs paper

In [11]:

```
df = pd.DataFrame(results).T
df = df.loc[[k for k in PAPER if k in df.index]]
df.to_csv(f'{OUT}/mantisv2_results_datasets_7_12.csv')
df[['ours_mean', 'ours_std', 'paper_mean', 'paper_std', 'diff', 'per_see
d', 'n_train', 'n_test', 'feat_dim', 'embed_s', 'rf_s']]
```

Out[11]:

	ours_mean	ours_std	paper_mean	paper_std	diff	per_seed
UCI-HAR	0.901256	0.001466	0.9013	0.002	-0.000044	[0.90329148 0.899898201 0.900...
Blink	0.995556	0.001814	0.9956	0.002	-0.000044	[0.99777777 0.995555555 0.993...
CAP-ID	0.809104	0.00104	0.8152	0.001	-0.006096	[0.81055654 0.808575955 0.808...
CAP-OOD	0.785804	0.001796	0.7859	0.0	-0.000096	[0.78511796 0.784029038 0.788...
Epilepsy-EEG	0.955779	0.00117	0.9558	0.001	-0.000021	[0.95420315 0.956129597 0.957...
FingerMovements	0.55	0.008165	0.55	0.01	0.0	[0.54, 0.56, 0